

Class 5

Wednesday, June 10, 2026

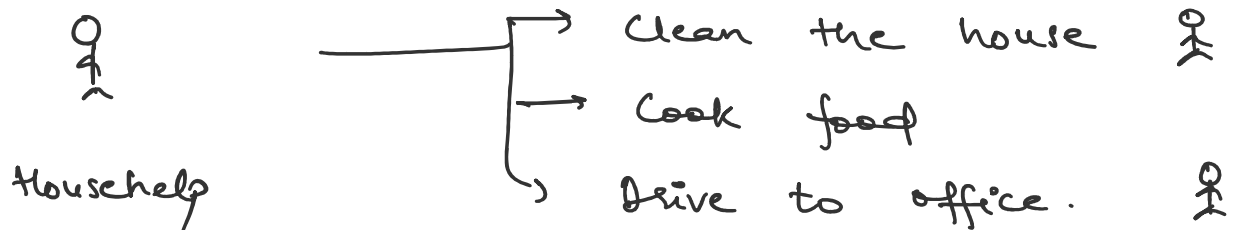
9:06 PM

S O L I D
↓ ↓ ↓ ↓ ↓

S → Single Responsibility Principle (SRP)

↓

One class should do only one thing.



↓

Too many responsibilities

↓

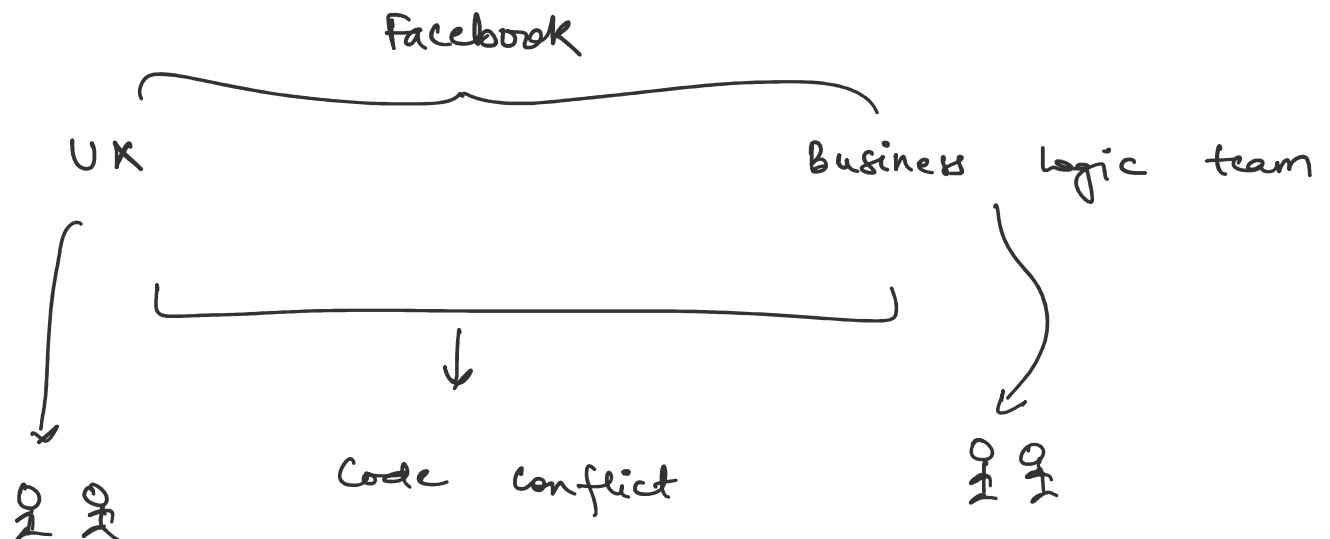
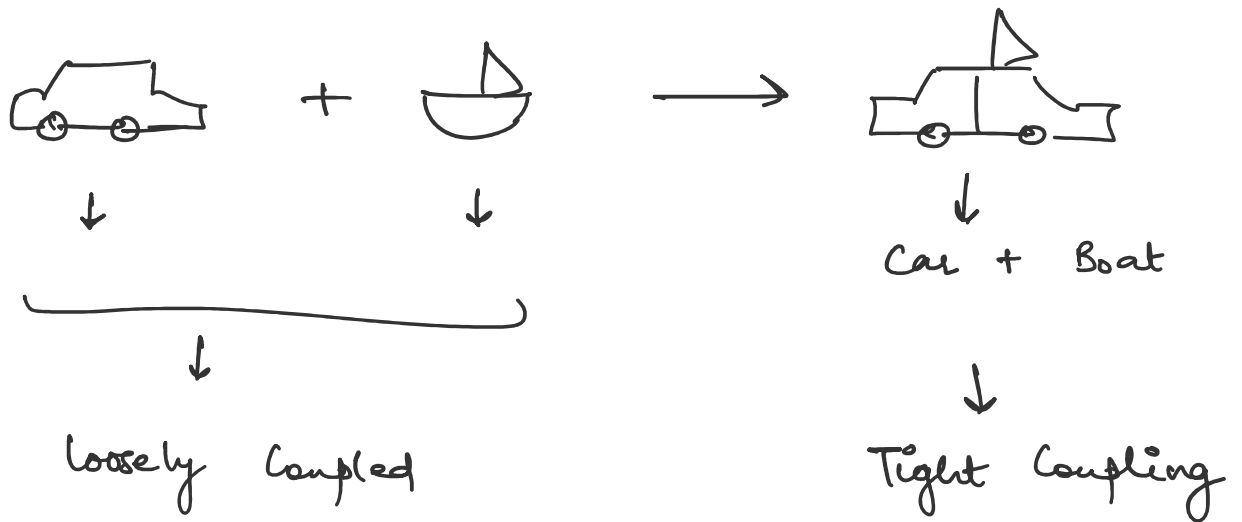
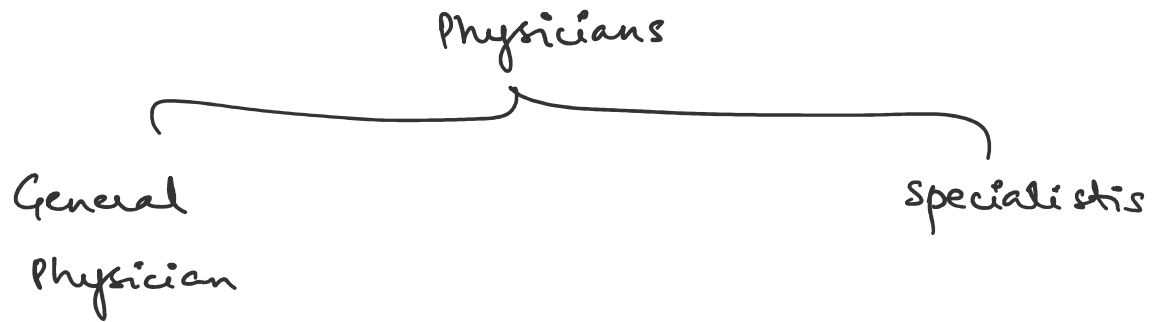
Messy code

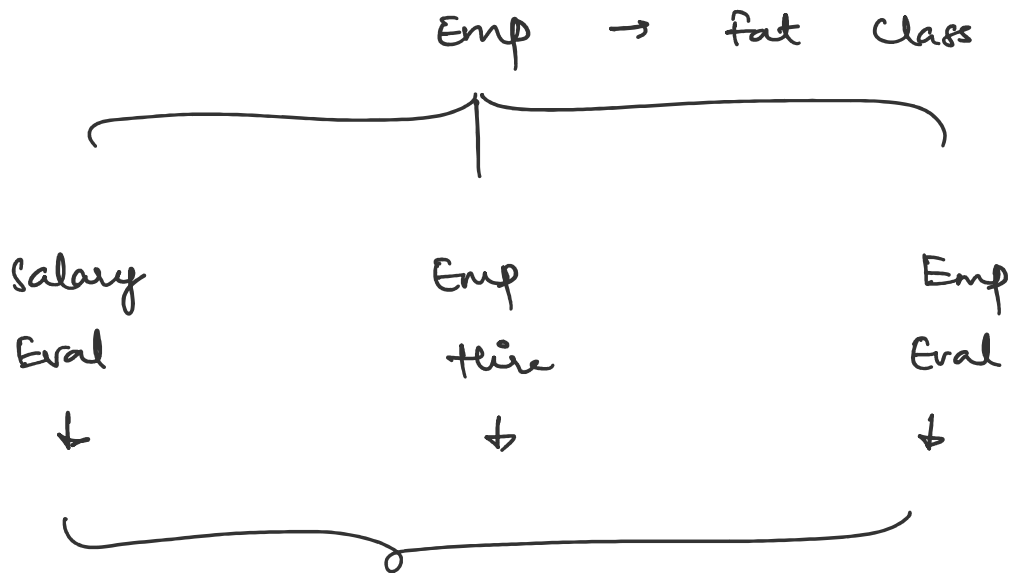
↓

Hard to maintain

Cook

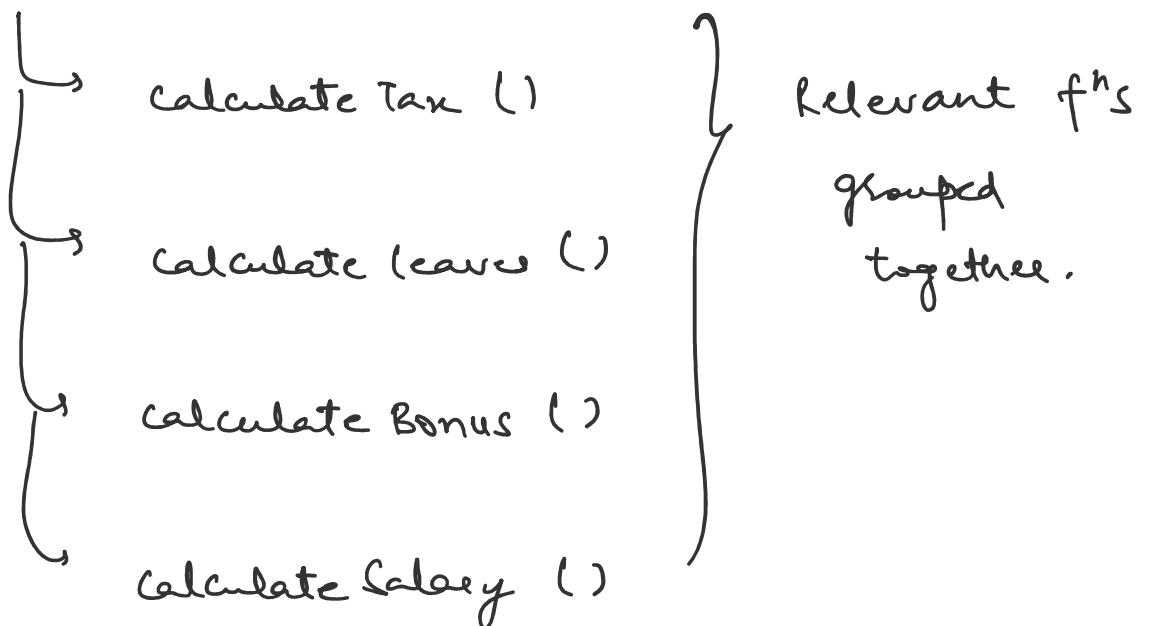
Eg - Hospital





SRP followed here

finance class



O → open close principle (OCP)

class

{
→ open for extension
→ closed for modification
}

Eg - Amazon → Paypal()

↓

Add support more payment opⁿ

class Payment

{

public void paypalPayment()

{

logic

}

{



To add support for more payment



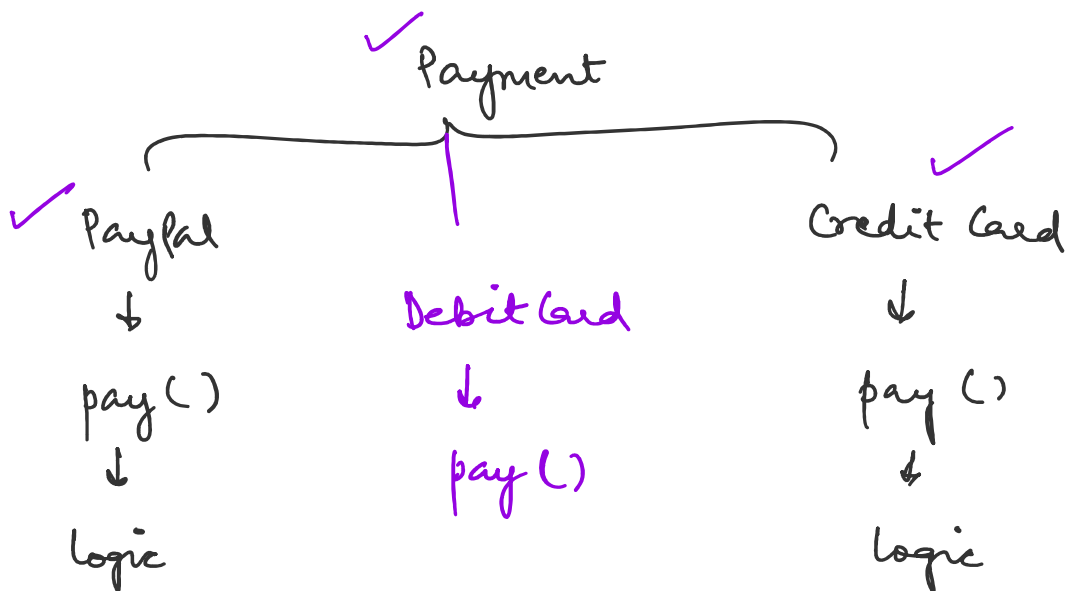
Not following OCP

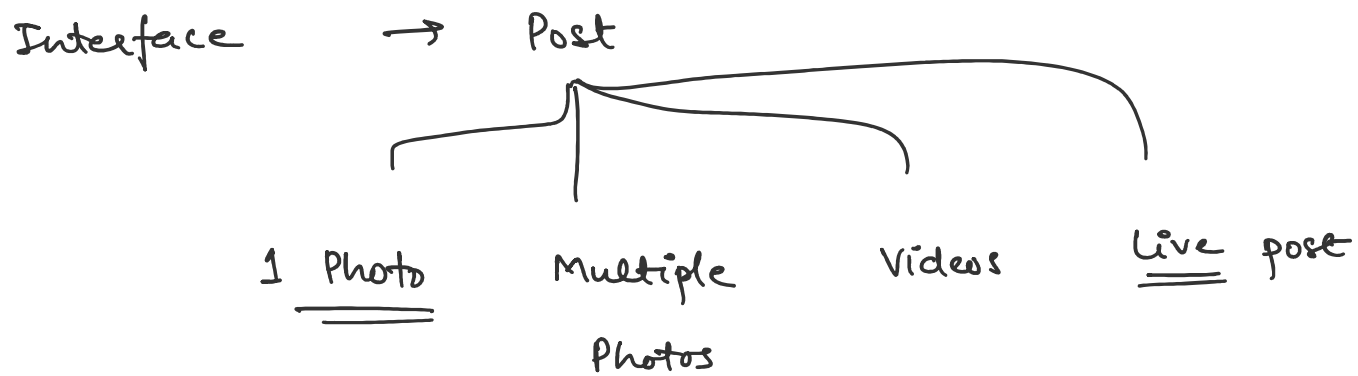
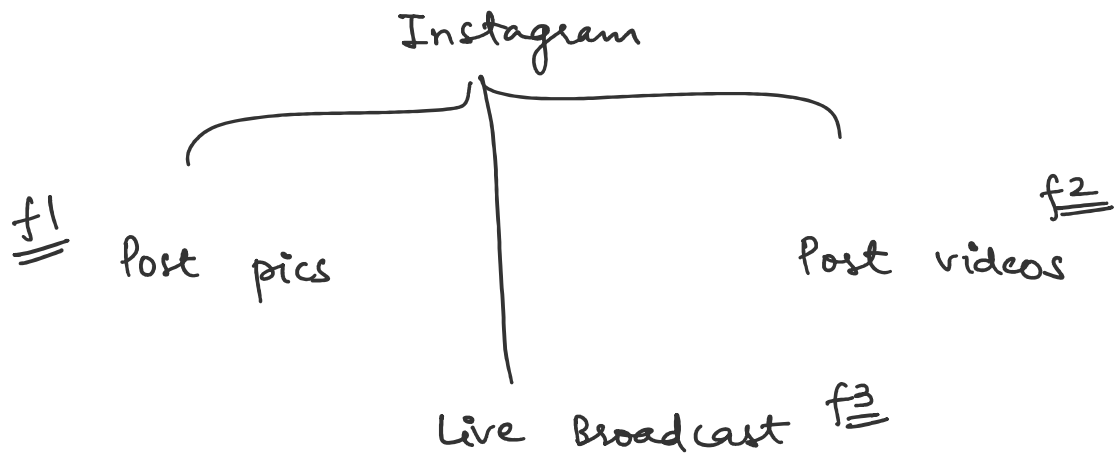
interface Payment

{

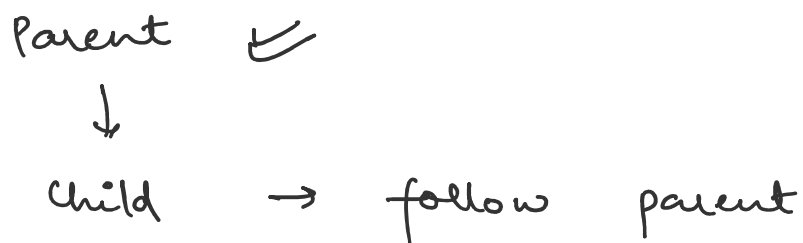
public void pay();

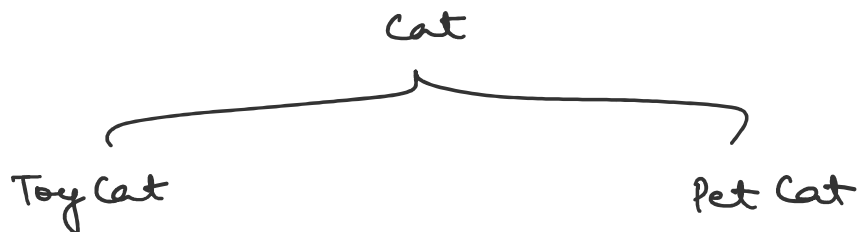
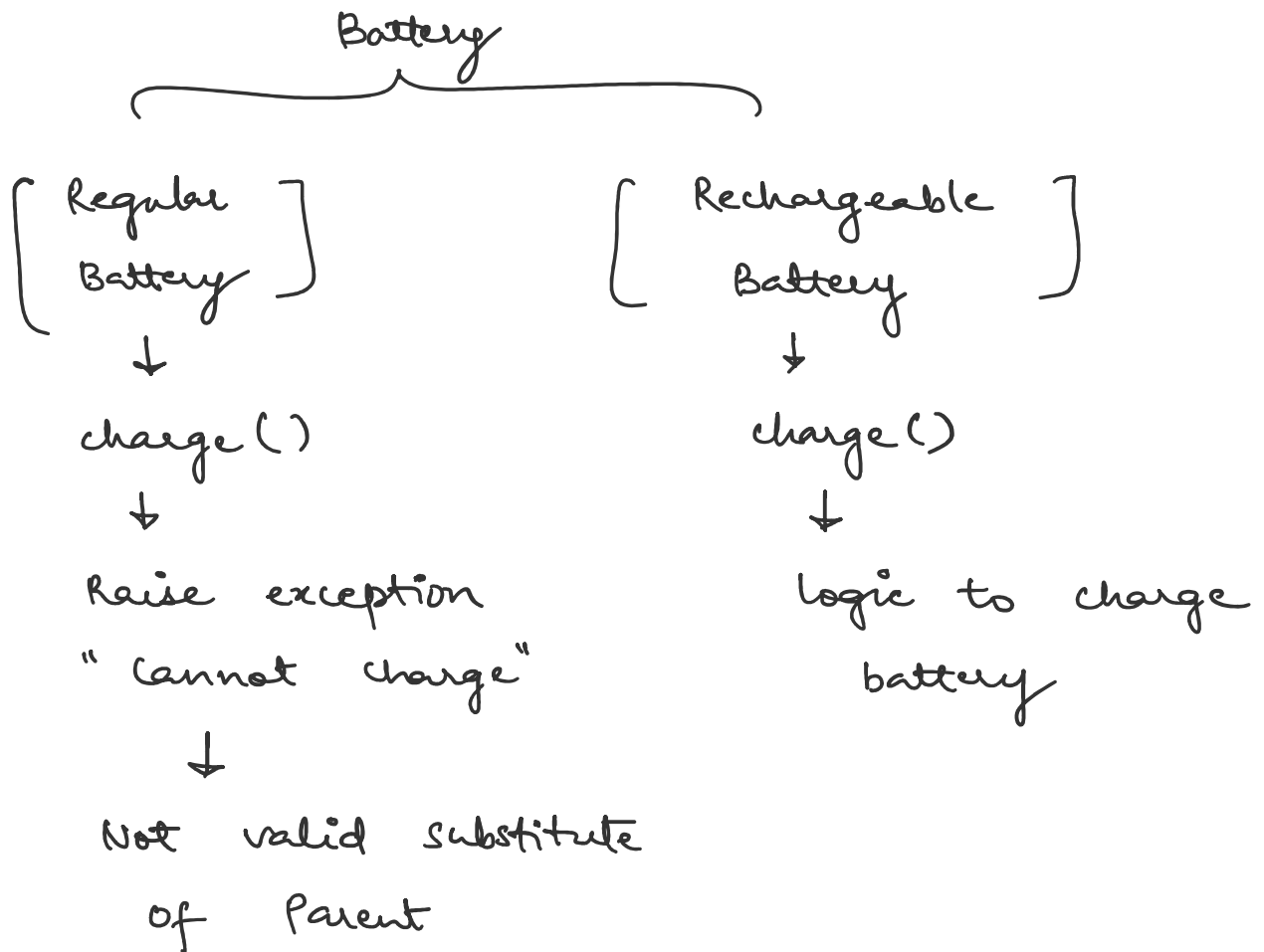
}





L - Liskov Substitution Principle (LSP)





```
interface Cat
{
    void drinkMilk();
    void speak();
}
```

}

```
class PetCat implements Cat  
{
```

drinkMilk() → "Cat is drinking
milk"

speak() → "Meow"

}

```
class ToyCat implement Cat  
{
```

drinkMilk() → Exception] x

speak() → "Meow"

}

I - Interface Segregation Principle (ISP)

```
interface cat
{
    void drinkMilk();
    void speak();
}
```

LSP, ISP x
Too many responsibilities



violates ISP

```
interface MechanicalCat
{
    public makeSound();
}
```

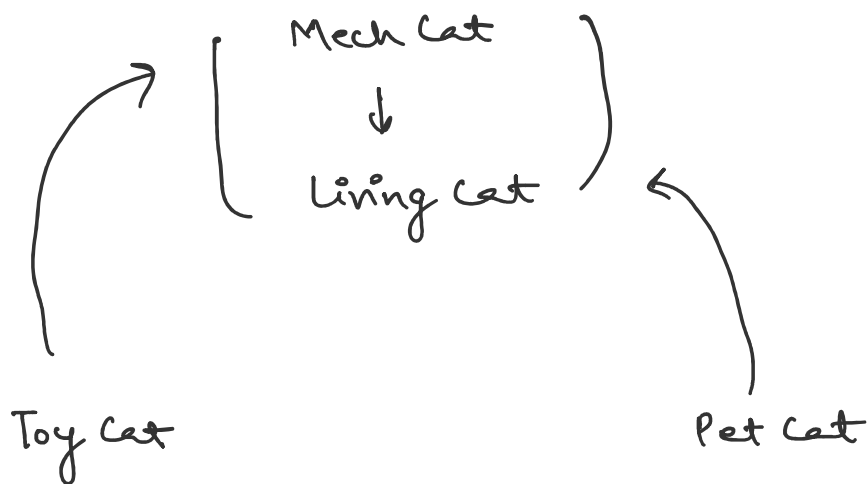
```
interface LivingCat extends MechCat
{
```

```
public    drink Milk ();
```

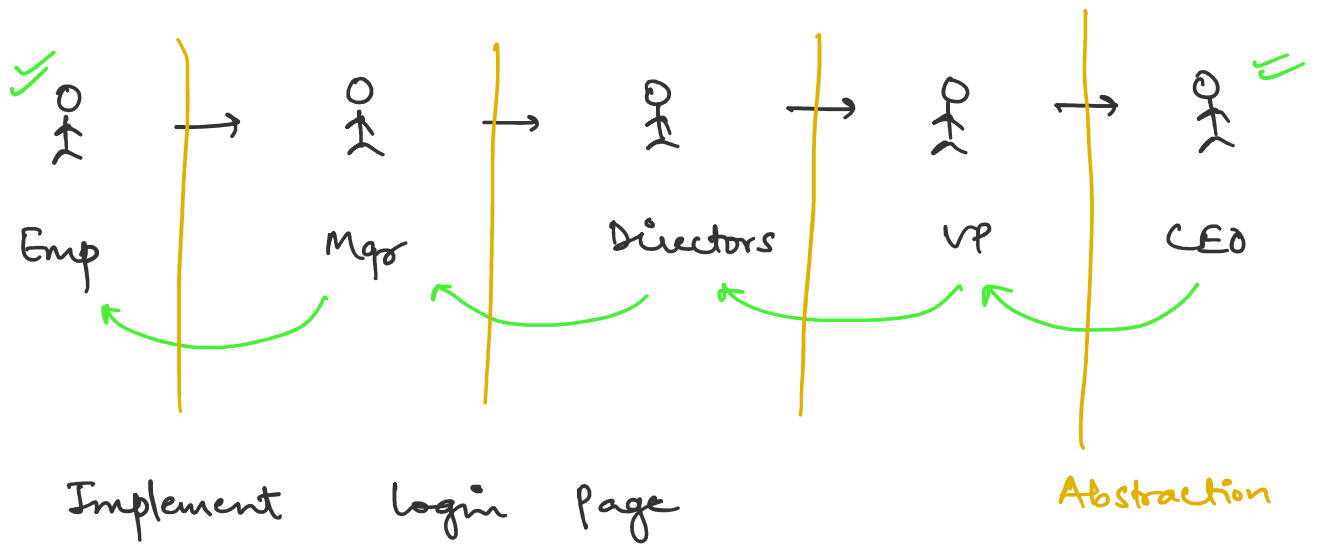
```
}
```

Don't force a class to implement fⁿs

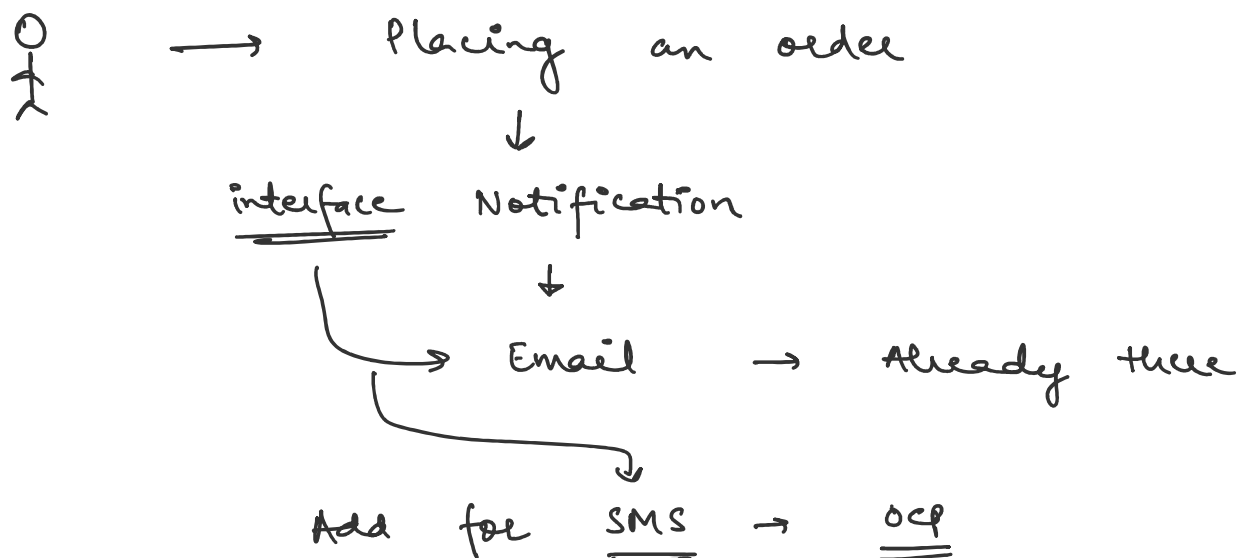
it does not need.



D - Dependency Inversion Principle (DIP)



High level modules should not depend upon low level modules. They should depend upon abstraction (interface).



Without DI -

```
class EmailService
{
    fn sendMail ()
}
```

```
class Notification
{
    private EmailService e ;

    Notification ()
    {
        e = new EmailService ();
    }

    public void send (String msg)
    {
        e.sendMail (msg)
    }
}
```

With DIP —

```
interface MsgService
{
    fn sendMail()
}
```

```
class EmailService implements MsgService
{
    ==
}
```

```
class SMSService implements MsgService
{
    ==
}
```

```
✓✓ class Notification
,
```

```

{
    private MsgService e;

    Notification()
    {
        e = new MsgService();
    }

    public void send (String msg)
    {
        e.sendMail (msg)
    }
}

```

Main Client Code

✓ SMS → Notification (SMS Service)

Email → Notification (Mail Service)

Notes :

S : Single Responsibility Principle (SRP)

Single Responsibility Principle (SRP) means **One class should do only one thing** and it **should have only one reason to change**.

The main goals of SRP are:

- Reduce confusion
- Reduce coupling
- Make classes more focused and specialized
- Make code easier to maintain and test

A class should be responsible for **only one job**, and **only one team/actor** should need to change it.

Real-Life Analogy:

A cook's job is to **cook food**, not:

- Clean the house
- Serve customers
- Manage payments

If one person does everything:

- Menu change → cook changes
- Cleaning rule change → cook changes again

Too many responsibilities = messy, hard to maintain code.

A class doing **too many things** becomes:

- Hard to understand
- Hard to test
- Risky to change

A class with **one clear responsibility** is:

- Easier to manage
- Easier to extend
- Safer to modify

When you put many components into one system, a failure in one part can affect the whole thing. This is the same idea behind the **Single Responsibility Principle** in software design. If one class tries to do too many things, and one part of it breaks, it can cause the whole system to fail. But if each class has one clear job, things stay simple, easy to manage, and more reliable. This concept is not just useful in software—it also applies to real life.

That's the core idea of the Single Responsibility Principle: **one class should only focus on one job or feature.**

Bad Design

```
import java.util.Date;

public class Employee {
    private int id;
    private String name;
```



```
private Date doj;
```

```
public Employee(int id, String name, Date doj) {  
    this.id = id;  
    this.name = name;  
    this.doj = doj;  
}
```

```
//Decided by company board of directors
```

```
public void calculateSalary() {  
    System.out.println("Logic for calculating salary.");  
}
```

```
//Decided by HR team
```

```
public void hireEmployee() {  
    System.out.println("Logic for hiring employee.");  
}
```

```
//Decided by Managers and evaluation team
```

```
public void evaluateEmployee() {  
    System.out.println("Logic for evaluating team.");  
}
```

```
public int getId() {  
    return id;  
}
```

```
public void setId(int id) {  
    this.id = id;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```

    }

    public Date getDoj() {
        return doj;
    }

    public void setDoj(Date id) {
        this.doj = doj;
    }
}

import java.util.Date;

public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee(1, "abc", new Date());
        System.out.println("Employee id : " + emp.getId()); //
Employee id : 1
        System.out.println("Employee name : " + emp.getName());
// Employee name : abc
        System.out.println("Employee date of joining : " +
emp.getDoj()); // Employee date of joining : Sun Nov 02
15:13:51 EST 2025
        emp.hireEmployee(); // Logic for hiring employee.
        emp.calculateSalary(); // Logic for calculating salary.
        emp.evaluateEmployee(); // Logic for evaluating team.
    }
}

```

The real meaning of SRP is this:

A class should only **change for one reason**, and it should only be

impacted by **one actor or team**.

Let's look at the class we're talking about:

- It has a *calculateSalary()* method – this logic is handled by the **finance team**.
- It also has a *hireEmployee()* method – this is something the **HR team** is responsible for.
- And it has an *evaluateEmployee()* method – this is the responsibility of the **manager**.

So, although it's just one class called Employee, **three different teams** are involved: finance, HR, and management.

That means if **any** of these teams need to make a change, they have to change this same class. And those changes can affect other parts of the class unintentionally.

For example: Let's say the **finance team** needs to update the tax calculation in *calculateSalary()*. Even though this has **nothing to do** with HR or employee evaluation, the whole class has to be **recompiled and redeployed**.

That also means while the finance changes are being made or deployed, the **HR and manager teams can't do anything**. They're blocked, even though their part of the code hasn't changed.

So what's the problem?

- Too many different responsibilities in one class.
- Too many teams working on the same class.
- One change affects everything else.
- It slows down development and increases risk.

This is why this class **violates the Single Responsibility Principle**.

And here's a simple way to remember:

Fat is not good for your health—and it's not good for classes either.

A **"fat class"** tries to do too much, and that leads to trouble.

Instead, each class should do just one job and be changed by just one team.

That's the true meaning of the Single Responsibility Principle.

Good Design - Following SRP

```
import java.util.Date;
```

//this is a class following SRP because it just contains the employee information

```
public class Employee {  
    private int id;  
    private String name;  
    private Date doj;
```

```
    public Employee(int id, String name, Date doj) {  
        this.id = id;  
        this.name = name;  
        this.doj = doj;  
    }
```

```
    public int getId() {  
        return id;  
    }
```

```
    public void setId(int id) {  
        this.id = id;
```

```

    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Date getDoj() {
        return doj;
    }

    public void setDoj(Date id) {
        this.doj = doj;
    }
}

public class SalaryCalculator {
    //Decided by company board of directors
    public void calculateSalary(Employee emp) {
        System.out.println("Logic for calculating salary.");
    }
}

public class EmployeeEvaluator {
    //Decided by Managers and evaluation team
    public void evaluateEmployee(Employee emp) {
        System.out.println("Logic for evaluating team.");
    }
}

public class EmployeeHiring {

```

```

//Decided by HR team
public void hireEmployee(Employee emp) {
    System.out.println("Logic for hiring employee.");
}
}

import java.util.Date;

public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee(1, "abc", new Date());
        System.out.println("Employee id : " + emp.getId()); //
        Employee id : 1
        System.out.println("Employee name : " + emp.getName());
        // Employee name : abc
        System.out.println("Employee date of joining : " +
        emp.getDoj()); // Employee date of joining : Sun Nov 02
        15:20:09 EST 2025

        SalaryCalculator salaryCalculator = new SalaryCalculator();
        salaryCalculator.calculateSalary(emp); // Logic for
        calculating salary.

        EmployeeHiring employeeHiring = new EmployeeHiring();
        employeeHiring.hireEmployee(emp); // Logic for hiring
        employee.

        EmployeeEvaluator employeeEvaluator = new
        EmployeeEvaluator();
        employeeEvaluator.evaluateEmployee(emp); // Logic for
        evaluating team.
    }
}

```

SRP says "Every software module should have only one reason to

change". This means that every class, or similar structure, in your code should have only one job to do. Everything in that class should be related to a single purpose. It does not mean that your classes should only contain one method or property. There may be many members as long as they relate to single responsibility.

O: Open closed Principle (OCP)

This principle says that our code should be **open for extension but closed for modification**. That means we should be able to **add new features** or **enhance** our program **without changing the existing code too much**.

This reduces:

- Bugs
- Risk
- Rework
- Breaking old functionality

Real-Life Analogy:

You can **plug in USB devices** to a laptop (extend functionality) without **modifying the laptop's internals**.

Another example :

Consider an application named *AnimalFeeder* that initially supports feeding only dogs. The application functions correctly and receives positive feedback from its users.

Later, a new requirement arises to support feeding cats as well. If the existing *AnimalFeeder* class is modified to incorporate cat-feeding functionality, the original codebase must be altered to accommodate the new feature. As additional animal types are introduced in the future, the same class would require repeated modifications.

This approach violates a key software design principle because continuously changing existing, working code increases maintenance effort, introduces the risk of new defects, and makes the system more difficult to extend over time.

A better design is to structure the application so that new functionality can be introduced by adding new components, such as classes that extend existing behavior, without modifying the already tested and stable code.

This design approach is known as the **Open/Closed Principle (OCP)**, which states that software entities should be **open for extension but closed for modification**.

```
class AnimalFeeder {  
    public void feedDog() {  
        System.out.println("feed Dog");  
    }  
}
```



```
    public void feedCat() {  
        System.out.println("feed Cat");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        AnimalFeeder animalFeeder = new AnimalFeeder();  
        animalFeeder.feedDog(); // feed Cat  
        animalFeeder.feedCat(); // feed Dog  
    }  
}
```

Initially, managing the code was straightforward. The *AnimalFeeder* application supported feeding only dogs, and the implementation was simple and effective.

As the application evolved, a new requirement emerged: support for feeding cats. To accommodate this, the existing *AnimalFeeder* class was modified to include cat-feeding functionality. At this stage, the change seemed reasonable and manageable.

However, consider a scenario where the application becomes highly successful and needs to support many different types of animals, such as rabbits, birds, elephants, and more.

With the current design, every time a new animal type is introduced, the *AnimalFeeder* class must be modified. Over time, this class will continue to grow, becoming increasingly complex and difficult to maintain. Each modification also introduces the risk of unintentionally affecting existing functionality.

The issue is not that the code will fail to compile or produce runtime errors. Rather, the design violates a fundamental software design principle known as the **Open/Closed Principle (OCP)**.

The Open/Closed Principle states that:

"Software entities should be open for extension but closed for modification."

In other words, we should be able to add new functionality without changing existing, stable code.

In our current implementation, adding support for a new animal requires modifying the *AnimalFeeder* class. Therefore, the design violates the Open/Closed Principle.

To address this problem, we can introduce an abstraction using an interface. Instead of placing all feeding logic inside a single class, we define a common contract that every animal type must follow. New animal types can then be added by creating new implementations of that interface, without modifying the existing *AnimalFeeder* code. This approach makes the system more scalable, maintainable, and extensible while adhering to the Open/Closed Principle.

```
interface Animal {  
    void feed();  
}
```

Now, for each animal, we create a separate class:

```
class Dog implements Animal {  
    @Override  
    public void feed() {
```

```
        System.out.println("Feeding dog...");
    }
}
```

```
class Cat implements Animal {
    @Override
    public void feed() {
        System.out.println("Feeding cat...");
    }
}
```

And now, our **AnimalFeeder** class doesn't care **which** animal it's feeding. It just calls the **feed()** method:

```
class AnimalFeeder {
    public void feedAnimal(Animal animal) {
        animal.feed();
    }
}
```

Now, if you want to add support for a **new animal**, like a rabbit, you just create a new class:

```
class Rabbit implements Animal {
    @Override
    public void feed() {
        System.out.println("Feeding rabbit...");
    }
}
```

No need to **touch or modify** the **AnimalFeeder** class. You just **extend** the behavior—**not modify existing code**.

This is how the **Open/Closed Principle** is applied in real-world coding using **interfaces** (or abstract classes). It makes your code more **flexible, scalable**, and **safe** from unwanted side effects when changes are made.

Main method :

```
public class Main {  
    public static void main(String[] args) {  
        AnimalFeeder animalFeeder = new AnimalFeeder();  
        animalFeeder.feedAnimal(new Dog()); // Feeding dog...  
        animalFeeder.feedAnimal(new Cat()); // Feeding cat...  
        animalFeeder.feedAnimal(new Rabbit()); // Feeding rabbit...  
    }  
}
```

L: Liskov Substitution Principle (LSP)

If a program works with a **parent class**, it should also work with any of its **child classes**—without breaking or behaving strangely.

In short you should be able to **replace a parent object with a child object** and the program should **continue to work normally**.

Think of it like this: Imagine you have a machine that needs a **Battery** to run. You use a basic **Battery** and everything works.

Now, someone gives you a **RechargeableBattery** (a child class of Battery). If you plug it in, the machine should still work **just like before**, or maybe even better—but it **shouldn't break** or act in an unexpected way.

That's **LSP**: The **RechargeableBattery** should behave like a proper **Battery**.

A child class **must NOT**:

- Change the meaning of parent behavior
- Disable expected functionality
- Throw unexpected errors
- Add side effects the parent never had

If it does → it is **not a valid substitute**.

In simple terms, **the child class must be a proper replacement for the parent class**. It should not break or change the expected behavior of the parent.

LSP is an **extension of the Open/Closed Principle**. It means when you create new subclasses, they should extend the parent class **without modifying its behavior**. LSP is a **safety rule** for OCP.

In many cases, developers blindly use inheritance by just checking if one class "**is a**" type of another. But **Liskov** says this is not enough. Just passing the "**is-a**" test doesn't mean the inheritance is correct.

Let's understand with a real-life example, just because someone is **born to a parent**, doesn't mean they automatically **inherit** the parent's hard-earned wealth.

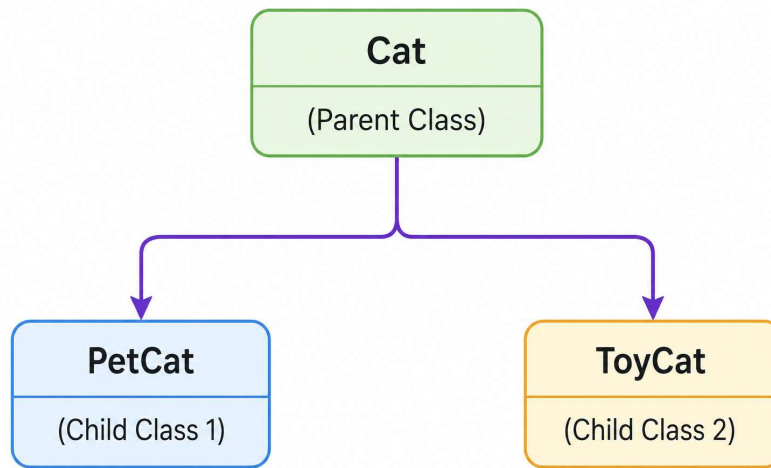
What matters is **how the child behaves**:

- If the child takes care of the parents,
- Respects their values,
- Acts responsibly

Only then, it feels right (both morally and socially) for the child to inherit what the parent owns.

But if the child ignores the parents, behaves badly, and disrespects them—**yet still expects to get everything the parents have**, we feel that's wrong.

This is exactly what **Liskov Substitution Principle** is about in software design.



“A toy cat *is a* cat, so it should inherit from Cat.”

Here **is-a** relationship is getting satisfied for inheritance, now we have to check it for Liskov’s Principle.

```
interface Cat {
    void drinkMilk();

    void speak();
}
```

Now we create a **ToyCat** class that implements Cat. But here’s the problem:

- **ToyCat** is a mechanical toy.
- If you call **drinkMilk()** on it, it **throws an exception** or says “*I don’t drink milk*”.

So even though **ToyCat** technically “is a” Cat (by inheritance), it **doesn’t behave like a real cat**. That means it **violates Liskov Substitution Principle**.

Because if we replace a real Cat with a **ToyCat**, the program will **break** or behave differently. That’s a sign that this inheritance is **not valid**.

Just like in real life, **a child should be a responsible and obedient child** to inherit from their parent. In software, a subclass should be a **proper substitute** for its parent class. Only then should inheritance be allowed.

If a subclass **changes** or **breaks** the expected behavior of the parent, it's not following the **Liskov** Substitution Principle, and that can lead to problems in the system.

I: Interface Segregation Principle (ISP)

Let's say we have a single interface like this:

```
interface Cat {  
    void drinkMilk();  
  
    void speak();  
}
```

Now, we create two types of cats:

- A **real cat** (can drink milk and make sound)
- A **toy cat** (can only make sound, cannot drink milk)

But the problem is, since both classes are forced to implement **Cat**, the **toy cat** must also implement the **drinkMilk()** method—even

though it doesn't make sense for a toy to drink milk.

So the toy cat might end up throwing an exception like:

```
public void drinkMilk() {  
    throw new UnsupportedOperationException("Toy cat cannot  
drink milk");  
}
```

Complete code :

```
public interface Cat {  
    void drinkMilk();  
  
    void speak();  
}
```

```
public class ToyCat implements Cat {  
    @Override  
    public void drinkMilk() {  
        throw new UnsupportedOperationException("Toy cat  
cannot drink milk.");  
    }  
  
    @Override  
    public void speak() {  
        System.out.println("Toy cat says: Meow!");  
    }  
}
```

```
public class PetCat implements Cat {
```

```

@Override
public void drinkMilk() {
    System.out.println("Pet cat drinks milk.");
}

@Override
public void speak() {
    System.out.println("Pet cat says: Meow!");
}
}

public class Main {
    public static void main(String[] args) {
        Cat petCat = new PetCat();
        petCat.drinkMilk(); // Pet cat drinks milk.
        petCat.speak(); // Pet cat says: Meow!

        Cat toyCat = new ToyCat();
        toyCat.speak(); // Toy cat says: Meow!
        toyCat.drinkMilk(); // Throws
        // UnsupportedOperationException
    }
}

```

This leads to **two major issues**:

- It **violates the Interface Segregation Principle** — because the interface has **more than one responsibility**, and forces the child class to implement things it doesn't need.
- It **violates the Liskov Substitution Principle** — because the toy cat **cannot fully behave like a real cat**, and throws exceptions where it should act normally.

To fix this, you break down the interface into **smaller, more specific interfaces**:

```
interface MechanicalCat {  
    void makeSound();  
}
```

```
interface LivingCat extends MechanicalCat {  
    void drinkMilk();  
}
```

Now:

- **ToyCat** can implement just **MechanicalCat**, and it's not forced to support drinking milk.
- **RealCat** can implement **LivingCat**, which includes both **makeSound()** and **drinkMilk()**.

Complete Code :

```
public interface MechanicalCat {  
    void speak();  
}
```

```
public interface LivingCat extends MechanicalCat {  
    void drinkMilk();  
}
```

```

public class ToyCat implements MechanicalCat {
    @Override
    public void speak() {
        System.out.println("Toy cat says: Meow!");
    }
}

```

```

public class PetCat implements LivingCat {
    @Override
    public void drinkMilk() {
        System.out.println("Pet cat drinks milk.");
    }

    @Override
    public void speak() {
        System.out.println("Pet cat says: Meow!");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        LivingCat petCat = new PetCat();
        petCat.drinkMilk(); // Pet cat drinks milk.
        petCat.speak(); // Pet cat says: Meow!

        MechanicalCat toyCat = new ToyCat();
        toyCat.speak(); // Toy cat says: Meow!
    }
}

```

This way:

- Each class only implements **what it truly needs**.
- There's no need for dummy implementations or exceptions.

- Now, if we replace the base class with a child class, everything behaves **as expected** — which means we're following the **Liskov Substitution Principle**.

Don't force a class to implement things it doesn't need.

Imagine you're filling out a form at the doctor's office. There's **one giant form** with questions for **everyone**: adults, kids, seniors, pregnant women, athletes, etc.

You're a healthy 25-year-old. But you still have to check boxes like:

- "Are you pregnant?"
- "Do you wear dentures?"
- "Do you have a pacemaker?"

That's **annoying** and **pointless** for you.

ISP says: Instead of **one big form**, make **smaller forms** for each type of patient:

- One for kids,
- One for seniors,
- One for athletes,
- etc.

This way, **everyone only fills out the form that's relevant to them.**

Another example : A printer should not be forced to implement fax functionality if it doesn't have that feature.

In Developer Terms: Don't create a **big interface** with **lots of methods** that all classes must implement. Instead, **split it up** into **smaller, focused interfaces**. Each class should only implement **what it actually needs**.

This helps in :

- Keeps your code **clean and focused**.
- Makes it **easier to understand, test, and change**.
- Avoids writing **dummy or empty methods** just to satisfy an interface.
- Each interface should have **one responsibility**, just like each class should. That's why ISP and SRP go hand in hand.

Bad Example — Violating ISP : Imagine a system for **machines**:

```
interface Machine {  
    void print(String doc);  
  
    void scan(String doc);  
  
    void fax(String doc);  
}
```

```
class MultiFunctionPrinter implements Machine {  
    public void print(String doc) {  
        System.out.println("MultiFunctionPrinter : Printing  
document : " + doc);  
    }  
}
```

```
    public void scan(String doc) {  
        System.out.println("MultiFunctionPrinter : Scanning  
document : " + doc);  
    }
```

```
    public void fax(String doc) {  
        System.out.println("MultiFunctionPrinter : Sending fax  
document : " + doc);  
    }  
}
```

```
class OldPrinter implements Machine {  
    public void print(String doc) {  
        System.out.println("OldPrinter : Printing document : " +  
doc);  
    }
```

```
    public void scan(String doc) {  
        throw new UnsupportedOperationException("Scan not  
supported");  
    }
```

```
    public void fax(String doc) {  
        throw new UnsupportedOperationException("Fax not  
supported");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Machine multiFunctionPrinter = new MultiFunctionPrinter();  
        multiFunctionPrinter.print("MyDocument.pdf"); //  
MultiFunctionPrinter : Printing document : MyDocument.pdf  
        multiFunctionPrinter.scan("MyDocument.pdf"); //  
MultiFunctionPrinter : Scanning document : MyDocument.pdf
```

```

    multiFunctionPrinter.fax("MyDocument.pdf"); //
    MultiFunctionPrinter : Sending fax document : MyDocument.pdf

    Machine oldPrinter = new OldPrinter();
    oldPrinter.print("OldDocument.pdf"); // OldPrinter :
    Printing document : OldDocument.pdf
    oldPrinter.scan("OldDocument.pdf"); // Throws
    UnsupportedOperationException
    oldPrinter.fax("OldDocument.pdf"); // Throws
    UnsupportedOperationException
}
}

```

Here **OldPrinter** doesn't support **scan()** or **fax()**, but it's **forced to implement them** anyway. That breaks the Interface Segregation Principle.

Good Example — Following ISP : We fix this by **splitting** the big interface into smaller, more specific ones.

```

interface Printer {
    void print(String doc);
}

```

```

interface Scanner {
    void scan(String doc);
}

```

```

interface Fax {
    void fax(String doc);
}

```



```
class OldPrinter implements Printer {  
    public void print(String doc) {  
        System.out.println("OldPrinter : Printing document : " +  
doc);  
    }  
}
```

```
class MultiFunctionPrinter implements Printer, Scanner, Fax {  
    public void print(String doc) {  
        System.out.println("MultiFunctionPrinter : Printing  
document : " + doc);  
    }
```

```
    public void scan(String doc) {  
        System.out.println("MultiFunctionPrinter : Scanning  
document : " + doc);  
    }
```

```
    public void fax(String doc) {  
        System.out.println("MultiFunctionPrinter : Sending fax  
document : " + doc);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        MultiFunctionPrinter multiFunctionPrinter = new  
MultiFunctionPrinter();  
        multiFunctionPrinter.print("MyDocument.pdf"); //  
MultiFunctionPrinter : Printing document : MyDocument.pdf  
        multiFunctionPrinter.scan("MyDocument.pdf"); //  
MultiFunctionPrinter : Scanning document : MyDocument.pdf  
        multiFunctionPrinter.fax("MyDocument.pdf"); //  
MultiFunctionPrinter : Sending fax document : MyDocument.pdf
```

```
OldPrinter oldPrinter = new OldPrinter();
oldPrinter.print("OldDocument.pdf"); // OldPrinter :
Printing document : OldDocument.pdf
    }
}
```

Benefits:

- No unnecessary or unsupported methods.
 - Each class depends only on what it truly needs.
 - More flexible and easier to maintain.
-

D: Dependency Inversion Principle (DIP)

Imagine you're directing a movie:

- You (the **Movie Maker**, high-level class) want to **make a movie**.
- You need: **Actors, Director, Producer, Choreographer**, etc. (these are low-level classes).

If you directly talk to and control each one (like *actor.act()*, *choreographer.dance()*), then your **Movie Maker** is **tightly coupled** to all of them.

Now imagine tomorrow you want to replace the **choreographer** or add a new one. You'd have to **change your Movie Maker class**. That **breaks the Open/Closed Principle (OCP)** too.

DIP Fix: Instead of depending on specific classes like *Choreographer*, *Director*, etc., the **Movie Maker** depends on **interfaces** like *Performer*, *CreativeRole*, etc.

That way:

- You can **easily plug in** new roles without touching the Movie Maker code.
- Your high-level logic stays the same.
- The system becomes **flexible and testable**.

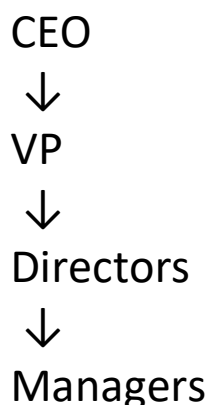
Dependency Inversion Principle (DIP) :

Core Rule:

- High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces).
- Abstractions should not depend on details. Details should depend on abstractions.

Another example :

"CEO of the company does not talk to the employees directly, he follows abstraction there."



↓
Employees

→ DIP followed here.

High-level modules (like the CEO) should not depend on low-level modules (like Employees). Both should depend on abstractions.

- In the example, the **CEO doesn't directly depend on Employees**.
- Instead, communication flows through layers (VP → Directors → Managers → Employees), representing **abstraction and decoupling**.
- Each layer depends on **interfaces or contracts**, not concrete implementations.

Another Example: Sending Messages

You want to build an app that **sends notifications** to users.

Without DIP (Bad Design — High-level depends on Low-level):

```
class EmailService {  
    public void sendEmail(String message) {  
        System.out.println("Email sent: " + message);  
    }  
}
```

```
class Notification {  
    private EmailService emailService;  
    public Notification() {  
        this.emailService = new EmailService();  
    }  
}
```

```

    }

    public void send(String message) {
        emailService.sendEmail(message);
    }
}

public class Main {
    public static void main(String[] args) {
        Notification notifyByEmail = new Notification();
        notifyByEmail.send("Your order has been shipped!"); //
        Email sent: Your order has been shipped!
    }
}

```

Problems:

- Notification (high-level class) **directly depends** on EmailService (low-level class). Notification is tightly bound to EmailService.
- You **can't switch** to SMS, WhatsApp, or push notifications **without changing the Notification class**. Hard to change or extend.
- This violates **DIP** and **Open/Closed Principle**.

With DIP (Good Design — Both depend on abstraction):

Step 1: Create an interface (abstraction)

```

interface MessageService {
    void sendMessage(String message);
}

```

Step 2: Implement different message services

```
class EmailService implements MessageService {  
    public void sendMessage(String message) {  
        System.out.println("Email sent: " + message);  
    }  
}
```

```
class SMSService implements MessageService {  
    public void sendMessage(String message) {  
        System.out.println("SMS sent: " + message);  
    }  
}
```

Step 3: High-level class depends on abstraction

```
class Notification {  
    private MessageService messageService;  
  
    public Notification(MessageService messageService) {  
        this.messageService = messageService;  
    }  
  
    public void send(String message) {  
        messageService.sendMessage(message);  
    }  
}
```

Step 4: Use the app

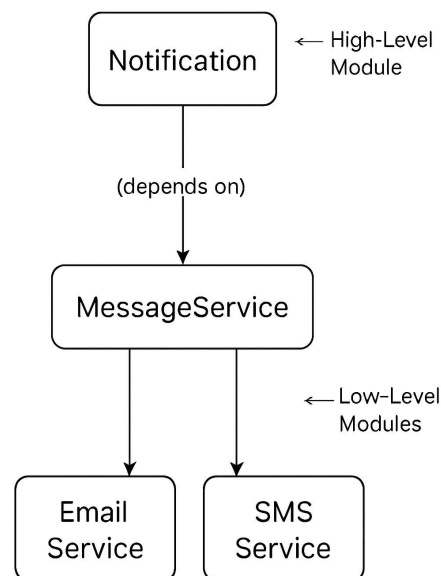
```

public class Main {
    public static void main(String[] args) {
        MessageService email = new EmailService();
        MessageService sms = new SMSService();

        Notification notifyByEmail = new Notification(email);
        notifyByEmail.send("Your order has been shipped!"); //
        Email sent: Your order has been shipped!

        Notification notifyBySMS = new Notification(sms);
        notifyBySMS.send("Your OTP is 123456"); // SMS sent:
        Your OTP is 123456
    }
}

```



With DIP

- Notification can use any message service

- Easy to add new types (like WhatsApp, Push)
- Follows both DIP and OCP

In Simple Words:

Don't build a class that directly depends on "**how**" something is done. Instead, depend on "**what**" needs to be done. Let someone else provide the "**how**."

In our case:

- **What** needs to be done? → Send a message.
- **How** it's done? → Email, SMS, WhatsApp, etc.

Why Is DIP Important?

- **Loosely Coupled Code:** Easy to modify and extend.
- **Testable Code:** You can inject mock implementations for testing.
- **Scalable:** Add new features (new coffee machines or movie roles) without changing existing logic.
- **Follows OCP & LSP:** Change or extend behavior through interfaces and subclasses without touching existing code.